

1. AutoTraq Development — Detailed Incident Report

BACKEND (API / Database / Services)

1. Backend Process Killed by Sub-Agent Tasks

What happened: During development, we used background AI sub-agents to build features in parallel (barcode system, part detail page, etc.). These sub-agents ran inside the same process tree as the backend server. When a sub-agent completed its task, the process cleanup sent a `SIGKILL` signal that cascaded up and killed the `tsx watch` process that was running the Express API server on port 3001. **How we discovered it:** Anson was actively using the app and suddenly got a white screen with "Failed to execute 'json' on 'Response': Unexpected end of JSON input" — the frontend was trying to call `/api/auth/me` but the backend was completely dead. No graceful error, no "server unavailable" message — just a raw JSON parse failure because the response was empty. **The fix:** Immediately ran `lsof -i :3001` to confirm the port was free, then restarted with `npm run dev`. Long-term, we created a root-level `package.json` using the `concurrently` package so both frontend and backend run in one terminal session with `npm run dev`, making restarts easier. We also became aware that sub-agent tasks needed to be isolated from the running server process. **Lesson learned:** Parallel development workflows need process isolation. A background build task should never share a process tree with a live server.

2. ---

2. Port 3001 EADDRINUSE — Zombie Node Processes

What happened: After the backend crashed and restarted multiple times, orphaned Node.js processes accumulated. At one point, `ps aux | grep tsx` revealed **three separate `tsx watch` processes** all trying to bind to port 3001. Only one could hold the port — the others were zombies consuming memory but unable to serve requests. New restart attempts would fail immediately with `EADDRINUSE`. **How we discovered it:** Tried to restart the backend and got `Error: listen EADDRINUSE: address already in use :::3001`. The normal `lsof -ti :3001 | xargs kill -9` didn't work on the first try because the shell's `lsof` wasn't resolving to the right path on macOS. **The fix:** Used the full path `/usr/sbin/lsof -i :3001` to find the exact PID holding the port, then `kill -9 <pid>`. After clearing the zombie, the fresh `npm run dev` started cleanly. We identified that the issue was caused by the `tsx watch` spawning child processes that persisted after the parent died. **Lesson learned:** On macOS, always verify which `lsof` you're calling. Zombie process

accumulation is a real risk when using file-watching dev servers with multiple restart triggers. ---

3. Wrong Startup Command — ts-node vs tsx

What happened: When the backend first went down, the initial restart attempt used `npx ts-node src/index.ts` instead of the project's configured `npm run dev` (which uses `tsx watch`). This immediately failed with `Cannot find module './routes/index.js'` because the codebase uses ESM-style `.js` extension imports in TypeScript files (e.g., `import routes from './routes/index.js'`). `ts-node` with its default CJS configuration couldn't resolve these — it looked for literal `.js` files that don't exist (only `.ts` files exist on disk).

3. How we discovered it: The error was immediate and clear — `MODULE_NOT_FOUND` with a require stack pointing at the import. **The fix:** Checked `package.json` scripts and found `"dev": "tsx watch src/index.ts"`. `tsx` handles ESM-style `.js` imports in TypeScript natively. Used the correct command going forward. **Lesson learned:** Always check the project's `package.json` scripts before improvising startup commands. TypeScript projects have specific toolchain requirements — `ts-node`, `tsx`, and `tsc + node` all behave differently with ESM imports. ---

4. Vehicle Search Returns Zero Results Despite 3,047 Vehicles

What happened: Anson typed "Ford" into the Vehicles search box and got "No vehicles found" despite the database containing 680+ Ford vehicles across all years. The search appeared completely broken. **Root cause analysis:** The search service used Prisma's `contains` filter on `make`, `model`, and `trim` fields. On MySQL, `contains` translates to `LIKE '%value%'` which is case-insensitive by default with the `utf8mb4_general_ci` collation. The query itself was correct. After extensive debugging — checking the database directly with raw MySQL queries (which returned Ford results), verifying the Vite proxy config, checking the API response format — the issue turned out to be that **Anson's locally running backend was still running old code** from before the search improvements were pushed. The `tsx watch` process had failed to pick up the git-pulled file changes because it was in a bad state from earlier crashes. **The fix:** Had Anson stop the backend completely (Ctrl+C), then restart fresh with `npm run dev`. The new process loaded the updated search service code and Ford immediately appeared. **Lesson learned:** File-watching dev servers can get into stale states after crashes and git operations. When search "doesn't work," always verify the running code matches the source files — a stale server is the most common cause.

4. ---

5. Partial Year Search — "201" Returns Nothing

What happened: Anson typed "201" into the vehicle search expecting to see all vehicles from 2010 through 2019. Instead: zero results. The search only understood exact 4-digit years. **Root cause:** The search logic parsed the input as an integer and checked `if (!isNaN(yearNum) && yearNum >= 1900 && yearNum <= 2100)`. The value `201` is a valid integer but it's not a valid year — it's 201 AD. So the year condition never matched, and "201" doesn't match any make, model, or trim as a text string either. **The fix:** Implemented partial year range matching. If the input is a number with fewer than 4 digits, pad it to create a range:

- "2" → 2000–2999 (too broad, but technically correct)
- "20" → 2000–2099
- "201" → 2010–2019
- "2014" → exact match 2014

Code: `const low = parseInt(token.padEnd(4, '0'), 10)` and `const high = parseInt(token.padEnd(4, '9'), 10)` → Prisma query `{ year: { gte: low, lte: high } }`. **Lesson learned:** Search UX needs to match how humans think.

Nobody types a full year all at once — they type character by character and expect progressive filtering at each keystroke.

5. ---

6. Multi-Word Search — "2014 ford" Shows All 2014 Vehicles

What happened: Anson typed "2014 ford" expecting to see only 2014 Ford vehicles. Instead, the results showed ALL 2014 vehicles — Buick, Cadillac, Chevrolet, everything. The "ford" part was completely ignored. **Root cause:** The search treated the entire input as a single string. `{ make: { contains: "2014 ford" } }` doesn't match anything because no make is literally named "2014 ford". The year condition matched `2014` from parsing the full string (JavaScript's `parseInt("2014 ford")` returns `2014` — it ignores trailing non-numeric characters). So only the year filter applied. **The fix:** Tokenized the search input by splitting on whitespace. For multi-word queries, each token generates its own set of OR conditions (could match year, make, model, or trim), then all tokens are combined with AND logic:

"2014 ford" → AND [

OR [year=2014, make LIKE '%2014%', model LIKE '%2014%'],

OR [make LIKE '%ford%', model LIKE '%ford%', trim LIKE '%ford%']

]

6. This means "2014 ford mustang" correctly narrows to: year 2014 AND make contains "ford" AND model contains "mustang". **Lesson learned:** Multi-field search with mixed data types (integers and strings) requires careful tokenization. The AND/OR pattern — AND between tokens, OR across fields within each token — is the standard approach for faceted search. ---

7. SKU Decode Failure for Manual SKUs

What happened: On the Scan page, looking up a manually-created SKU like "BRK-001" returned "Could not decode SKU" in red. The part existed in the database but the SKU decoder couldn't parse it because it didn't follow the auto-generated format. **Root cause:** The SKU lookup service expected the format `{2-char make}-{3-char model}-{2-digit year}-{2-char system}{2-char component}` (e.g., `FD-MUS-17-ENBR`). A manual SKU like "BRK-001" has a different structure — 3 chars, dash, 3 chars — which fails the parser's length and segment checks. **The fix:** On the Scan page, changed the lookup flow: first search for the part by exact SKU match in the database. If found, navigate directly to the part detail page. Only attempt SKU decoding as a fallback for parts not in the database. Added clear messaging: "This SKU was decoded but no matching part exists in inventory" vs "No part found and could not decode SKU." **Lesson learned:** Don't assume all data follows the same format. The system needs to handle both auto-generated and manually-entered identifiers gracefully. ---

7. FRONTEND (UI / UX / Components)

8. Magnifying Glass Icon Overlapping Search Text (Recurring)

What happened: The search input had a magnifying glass icon positioned absolutely inside the left side of the input field. As users typed, their text would render underneath the icon, making both unreadable. This was reported and "fixed" **three separate times** before being fully resolved. **Why it kept coming back:**

- First fix: Added `pointer-events-none` to the icon and increased `pl-14` padding. But the padding wasn't enough for longer text.
- Second fix: Increased padding further to `pl-14` with `pr-6`. But the icon was still visually competing with the text.
- Third fix (final): Removed the magnifying glass icon entirely. The placeholder text "Search by year, make, or model..." already communicates the purpose. Clean input, no overlap, no ambiguity.

Lesson learned: Decorative icons inside input fields are a UX liability. If they cause problems more than once, remove them. The placeholder text serves the same purpose without the positioning headaches. ---

8.

9. Button Text Clipping Outside Button Boundaries

What happened: The "Vehicle", "Group", and "New Part" buttons in the Parts page header had text that was visually touching or extending past the button edges. The buttons looked cramped and unprofessional — like the padding was calculated for shorter text. **Root cause:** Multiple compounding issues:

- `gap-2` (8px) between icon and text was too tight

- `px-4` (16px) horizontal padding wasn't enough for buttons with both an icon and text
- No `whitespace-nowrap` meant text could theoretically wrap on smaller viewports
- `flex` instead of `inline-flex` meant buttons could stretch based on parent container

The fix: Comprehensive button audit across ALL pages (PartsPage, VehiclesPage, InventoryPage, RequestsPage, PartDetailPage, ScanPage). Applied consistent styling:

- `inline-flex` instead of `flex`
- `gap-3` (12px) between icon and text
- `px-6` or `px-7` (24-28px) horizontal padding
- `py-3.5` (14px) vertical padding
- `whitespace-nowrap` to prevent text wrapping
- `rounded-xl` for consistent border radius

9. **Lesson learned:** Button styling needs to be defined as a design system token, not ad-hoc per component. When you fix a button style, grep the entire codebase and fix ALL buttons — partial fixes create inconsistency. ---

10. Layout Too Cramped — No Visual Breathing Room

What happened: Anson sent a screenshot showing the app looking squished — content jammed against the left side, minimal spacing between elements, sidebar barely distinguishable from the main content area. His exact feedback: "Everything looks crappy, its all squished with no proper padding and spacing." **Root cause:** The original layout used conservative spacing inherited from a mobile-first mindset:

- Sidebar: `w-64` (256px) with `p-4` internal padding
- Content area: `max-w-6xl mx-auto px-4 pt-6 pb-8`
- Cards: `rounded-xl p-6`
- Tables: `px-6 py-3.5` cells

On a large monitor (which Anson was using at the shop), these values made everything look miniature and cramped with massive empty space on the right. **The fix (iterative):** This required three rounds of adjustments:

10.
 - Round 1: Sidebar → `w-72`, content → `px-16 py-10`, cards → `rounded-2xl p-8`
 - Round 2: Sidebar → `w-[280px]`, content → `p-10 lg:p-12`, removed `max-w` constraint
 - Round 3: Table cells → `px-8 py-5` headers, button padding increases

Lesson learned: Desktop-first applications need desktop-first spacing. Don't start with mobile padding and hope it scales up — it won't. For internal business tools used on large monitors, generous whitespace isn't wasted space; it's readability. ---

11. Camera Scanner Fails on Desktop (No Webcam)

What happened: Anson plugged in a USB barcode scanner at the shop, went to the Scan page, and clicked "Start Camera Scanner." The app threw "Could not access camera" because the shop Mac mini doesn't have a webcam. The USB scanner and camera scanner are fundamentally different input methods, but the UI didn't make this distinction clear. **Root cause:** The scan page was originally designed for mobile use (phone camera). The USB barcode scanner acts as a keyboard HID device — it types characters rapidly and sends Enter. There was no UI pathway for USB scanner input; the only visible option was the camera button. **The fix:** Added a dedicated "USB Barcode Scanner" section with an "Enable Scanner Mode" toggle. When active:

- Listens for rapid global keystrokes (the USB scanner's HID input)
- Shows a pulsing "Listening for scanner input..." indicator

11.

- Auto-captures the scanned characters into a buffer
- Submits automatically when Enter is received (end of barcode)
- Navigates directly to the part detail page if found

Lesson learned: Know your deployment environment. A shop floor app needs to support shop floor hardware. USB barcode scanners are standard in inventory management — the app should have been designed for them from day one. ---

12. Sidebar Sub-Navigation Links Non-Functional

What happened: The collapsible sidebar had sub-items like "New Part" under Parts, "Search Vehicles" under Vehicles, etc. Clicking them did absolutely nothing — the URL changed to include query parameters (e.g., `/parts?action=new`) but the page didn't respond. **Root cause:** The sidebar was built with `NavLink` components that navigated to URLs with query parameters, but the page components never read those parameters. `PartsPage` didn't import or use `useSearchParams` from React Router, so `?action=new` was completely ignored. **The fix:** Added `useSearchParams` hook to each page component. On mount:

- `?action=new` → opens the create modal (`setShowPartModal(true)`)
- `?focus=search` → focuses the search input (`searchInputRef.current?.focus()`)

12.

- On Scan page: `?mode=usb` activates USB scanner mode, `?mode=camera` starts the camera

Lesson learned: When building navigation, test the full round-trip: click link → URL changes → page responds. A link that changes the URL but doesn't trigger any behavior is worse than a broken link — it looks like it works but doesn't. ---

13. Parts Table Rows Not Clickable

What happened: The Parts Catalog showed a table of all parts with their SKU, name, barcode, etc. But clicking on a row did nothing. There was no way to navigate

to the Part Detail page from the catalog — you had to manually type `/parts/1` in the URL bar. **Root cause:** The Part Detail page (`/parts/:id`) existed and was routed in `App.tsx`, but the table rows in `PartsPage` were plain `<tr>` elements with no click handler or navigation. **The fix:** Added `useNavigate` from React Router. Made each `<tr>` clickable with `onClick={() => navigate(`/parts/${part.id}`)}` and `cursor-pointer` class. Had to also add `e.stopPropagation()` on the action buttons (+ Fitment, + Group) so clicking those didn't trigger row navigation. **Lesson learned:** If you build a detail page, make sure users can actually get to it. Every list item should be clickable — it's the most basic navigation pattern in any CRUD application. ---

13.

14. Inconsistent Styling Across Pages

What happened: As features were built rapidly, each page ended up with slightly different styling:

- Parts page: `text-3xl` header, `rounded-2xl` cards, `px-8` table cells
- Vehicles page: `text-2xl` header, `rounded-xl` cards, `px-6` table cells
- Inventory page: `text-2xl` header, `rounded-lg` buttons, `px-4` padding
- Requests page: mixed styles from the original template

The app looked like it was designed by different people with different style guides.

The fix: Multiple sweep passes through all pages, grep-replacing inconsistent values:

- All headers → `text-3xl font-bold`
- All cards → `rounded-2xl`
- All buttons → consistent `px-6 py-3.5 rounded-xl gap-3`
- All table cells → `px-8 py-5` headers, `px-8 py-4` body
- All inputs → `px-6 py-4 rounded-2xl`

14. **Lesson learned:** Rapid prototyping creates style debt. Either build a component library / design tokens up front, or budget time for a consistency sweep. For a presentation-ready app, the sweep is non-negotiable. ---

DEVOPS / INFRASTRUCTURE

15. No Single Development Command

What happened: To run the app, developers had to open two separate terminal windows, navigate to two different directories, and run two different commands. New developers (or Anson switching contexts) had to remember: terminal 1 → `cd backend && npm run dev`, terminal 2 → `cd frontend && npm run dev`. If either terminal was accidentally closed, half the app went down. **The fix:** Created a root-level `package.json` with the `concurrently` npm package:

```
"dev": "concurrently -n api,web -c cyan,magenta \"npm run dev --prefix backend\" \"npm run dev --prefix frontend\""
```

Now one command (`npm run dev`) starts both, with color-coded output labels. Ctrl+C kills both.

15. Lesson learned: Developer experience is product quality. If it's annoying to start the app, developers will make mistakes. One command to rule them all. ---

16. Frontend Proxy Errors (ECONNREFUSED)

What happened: Anson started the frontend but forgot to start the backend. The Vite proxy to `/api` returned `AggregateError [ECONNREFUSED]` repeatedly in the terminal. The login page showed a cryptic JSON parse error instead of a helpful "server unavailable" message. **Root cause:** Vite's proxy silently fails when the target is unreachable. The frontend's `fetch('/api/auth/me')` got back an error response that wasn't valid JSON, causing `response.json()` to throw "Unexpected end of JSON input." **The fix:** The root `npm run dev` with `concurrently` prevents this by always starting both services together. The error handling in the API client could also be improved to catch non-JSON responses and show a "Backend server is not running" message. **Lesson learned:** Always handle the "backend is down" case in the frontend. Users will see proxy errors as app bugs, not infrastructure issues. ---

17. Hot Module Reload Not Detecting Git-Pulled Changes

16. What happened: After Anson ran `git pull` to get the latest code, the running `tsx watch` and Vite dev server didn't pick up all the changes. The backend was serving stale code — search logic that should have been updated was still running the old version. **Root cause:** `tsx watch` uses file system events (fsnotify) to detect changes. Git operations write files in bulk, and on macOS, the file system event queue can coalesce or drop events when many files change simultaneously. Additionally, if the `tsx watch` process was in a crashed/restarting state during the git pull, it could miss the events entirely. **The fix:** Manual restart after git pull: stop the server (Ctrl+C), then `npm run dev` again. This is a known limitation of file-watching tools. **Lesson learned:** After pulling significant code changes, always restart dev servers manually. Don't trust hot reload for git operations — it's designed for single-file edits during active development. ---

DATABASE

18. Vehicle Database Nearly Empty — Only 1 Vehicle

What happened: The Vehicles page showed a single 2026 vehicle. Searching for any year, make, or model returned either that one result or nothing. The entire vehicle dropdown system in the Parts creation modal was effectively useless — you couldn't assign parts to real vehicles because none existed.

17. Root cause: The vehicle seed script (`seed-vehicles.ts`) had been written but either never run, or run on a different database. The production database had

only one manually-created test vehicle. **The fix:** Ran the NHTSA (National Highway Traffic Safety Administration) API seed script, which:

- Queried the federal VPIC API for every US domestic make (Ford, Chevy, GMC, Dodge, Ram, Chrysler, Jeep, Lincoln, Cadillac, Buick, Tesla, etc.)
- For each make, queried every year from 2000-2026
- For each make/year, queried three vehicle types: Passenger Car, Truck, MPV
- Inserted 3,047 real vehicles with accurate year/make/model data
- Took ~10 minutes to run due to API rate limits

Lesson learned: Seed data isn't optional for demo/presentation apps. An inventory system with no vehicles is like a spreadsheet with no rows — technically functional, practically useless. Seed scripts should be part of the setup documentation and run automatically.

The “behind-the-scenes” part (the server)

- **A helper accidentally turned the server off.**

We used “robot helpers” to do jobs, but they were living in the same “house” as the server. When a helper finished, it cleaned up and accidentally shut off the whole server too. Then the app showed a white screen. We fixed it by restarting the server and making sure helpers don’t share the same “house.”

- **The server door (port 3001) was stuck “already in use.”**

Sometimes the server crashed and left “ghost servers” behind. Those ghosts still acted like they were using the door, so the real server couldn’t start. We found the ghost and kicked it out, then the server worked again.

- **We tried starting it the wrong way.**

We used the wrong start button, so the server couldn’t find its parts and wouldn’t run. We checked the right command and used that instead.

Searching for vehicles (cars) problems

- **Typing “Ford” said “no cars found,” even though there were lots.**

The search wasn’t really broken — the computer was running an *old* version of the code. We restarted it, and it worked.

- **Typing “201” found nothing.**

People type slowly, like “2...0...1...” but the app only understood full years like “2014.” We taught it that “201” should mean “2010 to 2019.”

- **Typing “2014 ford” showed every 2014 car, not just Ford.**

The app only listened to the “2014” part and ignored “ford.” We fixed it so it listens to *each word* and narrows down correctly.

Scanning part codes (SKU) problems

- **Some codes were “handmade” and the app didn’t understand them.**

The app expected a special code style, but some parts had simple codes like “BRK-001.” We fixed it so it first checks the database for an exact match, and only tries to “decode” if it’s not found.

The “what you see” part (the app screen)

- **The magnifying glass icon sat on top of the words you typed.**

It kept covering the text, so we finally removed the icon. The hint text already explains it’s for searching.

- **Buttons looked squished and messy.**

The words almost spilled out of buttons. We made button spacing bigger and made all buttons match everywhere.

- **Everything looked cramped on a big screen.**

On a large monitor, the app looked tiny and squeezed. We added more space and padding so it’s easier to read.

- **Camera scanning didn’t work on a computer with no camera.**

The shop computer didn’t have a webcam. But the barcode scanner is like a fast keyboard. We added a special mode for the USB scanner so it works correctly.

- **Sidebar links changed the page address but didn’t actually do anything.**

It looked like it worked, but nothing happened. We fixed pages so they actually listen to the “special instructions” in the URL.

- **You couldn't click a part in the table to open it.**

The detail page existed, but nothing took you there. We made rows clickable so you can open a part by clicking it.

- **Styles didn't match across pages.**

It looked like different people designed each page. We made all headings, buttons, cards, and tables use the same styles.

Setup problems (how we start the app)

- **You had to start two things separately.**

One for the front (website) and one for the back (server). Easy to forget. We made one simple command start both at the same time.

- **If the back wasn't running, the front showed confusing errors.**

We noted the app should show a nicer message like "The server isn't running."

- **After downloading new code, the running app sometimes didn't notice.**

So you might be using old code without knowing. Fix: restart after big updates.

Database problem (the car list was basically empty)

- **There was only 1 car in the database — that's why the car list was useless.**

We ran a big importer that grabbed real car info and loaded **3,047** vehicles, so the app has real data to use.